hspec: https://hackage.haskell.org/package/hspec
hspec-expectations: https://hackage.haskell.org/package/hspec-expectations
hspec manual: https://hspec.github.io/

```
- Test.Hspec
type Expectation = IO ()
hspec :: Spec -> IO ()
it :: (HasCallStack, Example a) => String -> a ->
SpecWith (Arg a)
xit :: (HasCallStack, Example a) => String -> a ->
SpecWith (Arg a)        -- ignore
fit :: (HasCallStack, Example a) => String -> a ->
SpecWith (Arg a)        -- focus i.e. opposite of ignore
describe :: HasCallStack => String -> SpecWith a ->
SpecWith a
xdescribe :: HasCallStack => String -> SpecWith a ->
SpecWith a
fdescribe :: HasCallStack => String -> SpecWith a ->
SpecWith a
focus :: SpecWith a -> SpecWith a
pending :: HasCallStack => Expectation
pendingWith :: HasCallStack => String -> Expectation
example :: Expectation -> Expectation
parallel :: SpecWith a -> SpecWith a

-- Hooks
type ActionWith a = a -> IO ()
before_ :: IO () -> SpecWith a -> SpecWith a
                      -- before each
before :: IO a -> SpecWith a -> Spec
                      -- IO a = run (SpecWith a) with
input a
beforeWith :: (b -> IO a) -> SpecWith a -> SpecWith b
beforeAll_ :: HasCallStack => IO () -> SpecWith a ->
SpecWith a                -- before group
beforeAll :: HasCallStack => IO a -> SpecWith a -> Spec

beforeAllWith :: HasCallStack => (b -> IO a) -> SpecWith
a -> SpecWith b

after_ :: IO () -> SpecWith a -> SpecWith a
                      -- after each
after :: ActionWith a -> SpecWith a -> SpecWith a
                      -- a -> IO () = cleanup a after
running (SpecWith a)
afterAll_ :: HasCallStack => IO () -> SpecWith a ->
SpecWith a                -- after group
afterAll :: HasCallStack => ActionWith a -> SpecWith a
-> SpecWith a

around_ :: (IO () -> IO ()) -> SpecWith a -> SpecWith
a                         -- before + after each
```

```
around :: (ActionWith a -> IO ()) -> SpecWith a ->
Spec                       -- a -> IO () = run (SpecWith
a) with input a
aroundWith :: (ActionWith a -> ActionWith b) -> SpecWith
a -> SpecWith b
aroundAll_ :: HasCallStack => (IO () -> IO ()) ->
SpecWith a -> SpecWith a  -- before + after group
aroundAll :: HasCallStack => (ActionWith a -> IO ()) ->
SpecWith a -> Spec
aroundAllWith :: HasCallStack => (ActionWith a ->
ActionWith b)
  -> SpecWith a -> SpecWith b
mapSubject :: (b -> a) -> SpecWith a -> SpecWith b
ignoreSubject :: SpecWith () -> SpecWith a

newtype SpecM a r = …
type SpecWith a = SpecM a ()
type Spec = SpecWith ()

type family Arg e
type Arg Property = ()
type Arg Expectation = ()

type Arg (a -> Property) = a
…


class Example e where
 type Arg e
 evaluateExample :: …
instance Example Property
instance Example Expectation
…
```

Expectations

```
-- Test.Hspec.Expectations
type Expectation = IO ()

expectationFailure :: HasCallStack => String ->
Expectation
shouldBe :: (HasCallStack, Show a, Eq a) => a -> a ->
Expectation
shouldSatisfy :: (HasCallStack, Show a) => a -> (a ->
Bool) -> Expectation
shouldStartWith :: (HasCallStack, Show a, Eq a) => [a]
-> [a] -> Expectation
shouldEndWith :: (HasCallStack, Show a, Eq a) => [a] ->
[a] -> Expectation
shouldContain :: (HasCallStack, Show a, Eq a) => [a] ->
[a] -> Expectation
shouldMatchList :: (HasCallStack, Show a, Eq a) => [a]
-> [a] -> Expectation
shouldReturn :: (HasCallStack, Show a, Eq a) => IO a ->
```

```
a -> Expectation
shouldNotBe :: (HasCallStack, Show a, Eq a) => a -> a ->
Expectation
shouldNotSatisfy :: (HasCallStack, Show a) => a -> (a ->
Bool) -> Expectation
shouldNotContain :: (HasCallStack, Show a, Eq a) => [a]
-> [a] -> Expectation
shouldNotReturn :: (HasCallStack, Show a, Eq a) => IO a
-> a -> Expectation

type Selector e = e -> Bool

shouldThrow :: (HasCallStack, Exception e) => IO a ->
Selector e -> Expectation
anyException :: Selector SomeException
                                  -- ... `shouldThrow`
anyException
anyErrorCall :: Selector ErrorCall
                                  -- ... `shouldThrow`
anyErrorCall     Failure by "error" function
anyIOException :: Selector IOException
anyArithException :: Selector ArithException
errorCall :: String -> Selector ErrorCall
                                  -- ... `shouldThrow`
errorCall "ERROR"
```

Types /classes
- type **Spec = SpecWith ()**: a single test or a group of tests
- type **SpecWith a**: a single test or a group of tests with **test params** aka arguments a
  i.e. a test with **setup** parameters, created by setup hooks
  e.g. by the **before** hook
- **class Example**: the content of a single test.
  e.g. the **Exception type = Assertion = IO ()** representing an assertion.
  e.g. the **Property type** representing a single property test

e.g. relationship of types

```
describe "My tests" $ do                              --
describe :: String -> Spec
  -- tests with no args
  describe "Basic" $ do                               --
describe :: String -> Spec
    it "1 == 1" $                                     --
it :: String -> Expectation -> Spec
      1 `shouldBe` 1

  -- tests with args
  testSetup `before` $                                  --
before :: IO Int -> SpecWith Int -> Spec
    testCleanup `after` $                               --
after :: ActionWith Int -> SpecWith Int -> SpecWith Int
      describe "Tests with test args" $ do            --
```

```
describe :: String -> SpecWith Int -> SpecWith Int
        it "is not zero" $ \v -> do                  --
it :: String -> (Int -> Expectation) -> SpecWith (Arg
(Int -> Expectation))
          v `shouldNotBe` 0
  where
    testSetup :: IO Int
    testSetup = getStdRandom uniform
    testCleanup v = putStrLn $ "tested with=" ++ show v
```

CLI
Ref: https://hspec.github.io/options.html

```
# Running
#
# With bin
$ ./test --randomize --fail-fast

# With Stack
$ stack test <target> --test-arguments="--randomize --
fail-fast"


# Basic
$ <CLI> --help
$ <CLI> --randomize …
$ <CLI> --dry-run …
$ <CLI> --fail-fast …

$ <CLI> --match "hello"
$ <CLI> -m "hello"
$ <CLI> -m "*"            # NO wildcard?
$ <CLI> -m "*"            # NO regex?
```

Usage: run tests with arguments in GHCI
Passing options: https://hspec.github.io/options.html
Running individual tests: https://hspec.github.io/match.html
Use **:main** command

e.g. in Stack

```
-- ./test/Spec.hs
import Hspec

main = describe "My test" $ do
  …
```

Run in GHCI

```
$ stack ghci MyProject:test:main-test
```

```
ghci> :main --match "Test1"
ghci> :main -m "Test1"             -- same
```

Run with stack

```
$ stack test --test-arguments=" --match='Test1'"
MyProject:test:main-test
```

Usage: Basic setup
Ref: https://hspec.github.io/writing-specs.html

e.g.

```
import System.Random
import Test.Hspec

main = hspec mainSpecs

mainSpecs :: SpecWith ()
mainSpecs =
  -- e.g. hooks
  beforeAll_ onBefore $
    afterAll_ onAfter $
      before_ onBeforeEach $
        after_ onAfterEach specs1
  where
    onBefore = putStrLn "Before"
    onAfter = putStrLn "After"
    onBeforeEach = putStrLn "BeforeEach"
    onAfterEach = putStrLn "AfterEach"

randomInt :: IO Int
randomInt = getStdRandom uniform

specs1 :: SpecWith ()
specs1 = describe "My tests" $ do
   -- e.g. basic test
  describe "Basic" $ do
    it "1 == 1" $
      1 `shouldBe` 1
    it "1 /= 2" $
      1 `shouldNotBe` 2

  -- e.g. test using IO
  describe "IO" $ do
    it "use IO" $ do            -- can use IO since
Expectation is just IO ()
      putStrLn "Enter 1"
      z <- getLine
      z `shouldBe` "1"
    it "Random" $ do
      x <- randomInt
```

```
      y <- randomInt
      x `shouldNotBe` y

  -- e.g. pending tests
  describe "Pending" $ do
    it "TODO 1" $ do
      pending
    it "TODO 2" $ do
      pendingWith "TODO"

  -- e.g. tests with setup values
  before testSetup $                                  --
before: create test params
    after testCleanup $                               --
after: cleanup test params
      describe "Tests with test args" $ do
        it "is not zero" $ \v -> do                   --
test with params
          v `shouldNotBe` 0
  where
    testSetup :: IO Int
    testSetup = getStdRandom uniform
    testCleanup v = putStrLn $ "tested with=" ++ show v
```

Output

```
$ ./main
My tests
  Basic
Before
BeforeEach
AfterEach
    1 == 1 [✔]
BeforeEach
AfterEach
    1 /= 2 [✔]
  IO
BeforeEach
Enter 1
1
AfterEach
    use IO [✔]
BeforeEach
AfterEach
    Random [✔]
  Pending
BeforeEach
AfterEach
    TODO 1 [-]
      # PENDING: No reason given
BeforeEach
AfterEach
    TODO 2 [-]
      # PENDING: TODO
```

```
  Tests with test args
BeforeEach
tested with=-1463817299972272612
AfterEach
    is not zero [✔]
After

Finished in 0.8575 seconds
7 examples, 0 failures, 2 pending
```

Usage: bootstrapping e.g. start/stop server
Use **aroundAll**

e.g. start/stop server

```
import Control.Exception (bracket)
aroundAll :: HasCallStack => (ActionWith a -> IO ()) ->
SpecWith a -> Spec
aroundAll_ :: HasCallStack => (IO () -> IO ()) ->
SpecWith a -> SpecWith a

specs :: Spec
specs = (\run -> do
  bracket
   startServer                    -- step: bootstrap
 startServer :: IO Server
   stopServer                     -- step: cleanup
 stopServer :: Server -> IO ()
   (const run)                    -- step: run test
 ) `aroundAll` baseSpecs


baseSpecs :: Spec
baseSpecs = describe "main" $ do
 it "should do something" $ do
  …
```

e.g. start server on a different thread

```
import Control.Exception (bracket)
import Control.Concurrent

oneSecond = 1000 * 1000

specs :: Spec
specs = (\run -> do
  bracket
   (forkIO startServer <* threadDelay oneSecond)
-- step: bootstrap  start on a different thread
 startServer :: IO Server

 --       Delay to wait until server is started
```

```
  killThread
 -- step: cleanup    kill server thread
  (const run)
 -- step: run test
 ) `aroundAll` baseSpecs


baseSpecs :: Spec
baseSpecs = describe "main" $ do
 it "should do something" $ do
  …
```

```
---- Test
it "test" $ do …                        -- Spec
it "test" $ \s -> do …                  -- Spec String
fit "test" $ do …                       -- focus i.e. only
run this test
focus $ it "test" $ do …                -- same
xit "test" $ do …                       -- ignore

-- Arguments
describe :: SpecWith a
describe "main" $ do                      -- Change
arguments
 it "test" $ \a -> do …                   -- SpecWith a
 ignoreSubject $ it "test" $ do …         -- Spec
 mapSubject f $ it "test" $ \b -> do …    -- SpecWith b


---- Expectation
--
1 `shouldBe` 1
1 `shouldNotBe` 1
1 `shouldSatisfy` even
io `shouldThrow` (\(e :: IOException) -> True)
io `shouldThrow` anyException
```

==========
======

Tasty

Hackage: https://hackage.haskell.org/package/tasty-1.4.2.3#readme

Tasty

```
type TestName = String
data TestTree

defaultMain :: TestTree -> IO ()
defaultMainWithIngredients :: [Ingredient] -> TestTree
-> IO ()
testGroup :: TestName -> [TestTree] -> TestTree
```

```
withResource :: IO a -> (a -> IO ()) -> (IO a ->
TestTree) -> TestTree

defaultIngredients :: [Ingredient]
includingOptions :: [OptionDescription] -> Ingredient
adjustOption :: IsOption v => (v -> v) -> TestTree ->
TestTree
localOption :: IsOption v => v -> TestTree -> TestTree
askOption :: IsOption v => (v -> TestTree) -> TestTree

data Timeout = Timeout Integer String | NoTimeout
mkTimeout :: Integer -> Timeout

data DependencyType = AllSucceed | AllFinish
after :: DependencyType -> String -> TestTree ->
TestTree
after_ :: DependencyType -> Expr -> TestTree -> TestTree
```

Tasty.Hspec

```
testSpec :: TestName -> Spec -> IO TestTree
testSpecs :: Spec -> IO [TestTree]
```

Tasty.Hedgehog

```
testProperty :: TestName -> Property -> TestTree
testPropertyNamed :: TestName -> PropertyName ->
Property -> TestTree
fromGroup :: Group -> TestTree
```

Usage: run with CLI arguments
e.g. in stack

```
$ stack --test-arguments "--pattern mytest"        # run
only tests containing
```